1.4

```cpp
#include <windows.h>
#include <iostream>
#include <stdexcept>
#include "my_queue.h"

using namespace std;

#define VISITORS_NUM 8
#define CHAIRS_NUM 2

/* 3
АДАЧА ПРО БАРБЕРА БЛА БЛА БЛА

Метод вирішення - Критичні секції
критичні секції використовуються для забезпечення правильної синхронізації доступу до спільних ресурсів з боку кількох потоків.
Вони гарантують, що тільки один потік може виконувати код у критичній секції одночасно, тим самим запобігаючи гонкам
за даними і іншим проблемам, пов'язаним з одночасним доступом до спільних ресурсів.
*/

/*
У цих рядках оголошуються змінні, які будуть використовуватися для синхронізації потоків і збереження даних.
visitors_queue - критична секція, яка використовується для синхронізації доступу до черги відвідувачів.
output - критична секція, яка використовується для синхронізації доступу до виводу на екран.
barber - критична секція, яка використовується для синхронізації доступу до барбера.
barber1 - критична секція, яка використовується для синхронізації доступу до барбера (використовується в основному потоці).
*/
CRITICAL_SECTION visitors_queue;
CRITICAL_SECTION output;
CRITICAL_SECTION barber;
CRITICAL_SECTION barber1;
```

```cpp
DWORD WINAPI functionBarber(LPVOID arg) //це функція, яка виконується в окремому потоці.Вона відповідає за роботу барбера.
{
    while (true) // постійний цикл, тобто якщо в мене черга буде пуста, там парбер буде спати
                 // ну очікувати нових відвідувачей
    {
        EnterCriticalSection(&visitors_queue); // вхід до критичної секції visitors_queue
                                               // тобто я отримаю доступ до черги відвідувачів
        // далі в мене йде перевірка на порощню чергу
        if (visitors->getSize() == 0) // якщо так, то барбер має спати
        {
            LeaveCriticalSection(&visitors_queue); // вихід з критичної секції visitors_queue
            EnterCriticalSection(&output); // вхід до критичної секції виводу на екран ну і власне вивів повідомлення
            cout << "Barber is sleeping" << endl;
            LeaveCriticalSection(&output); // виходить з критичної секції output.

            Sleep(1000);
            EnterCriticalSection(&barber1); // входить в критичну секцію barber1, щоб заставити барбера чекати на клієнта.
        }
        else // У протилежному випадку черга не є порожньою і барбер працює з клієнтом.
        {
            int clientId = visitors->pop(); // я отримую ідентифікатор клієнта з черги.
            LeaveCriticalSection(&visitors_queue); //виходить з критичної секції visitors_queue.

            Sleep(2000 + rand() % 2000); // імітація роботи з клієнтом

            EnterCriticalSection(&output); //входить в критичну секцію output, щоб забезпечити доступ до виводу на екран.
            cout << "Barber done work and client " << clientId << " go away" << endl; // виводжу повідомлення
            LeaveCriticalSection(&output); // вихід з критичних секцій output та barbeer
            LeaveCriticalSection(&barber);
        }
    }
    return true;
}
```

```cpp
DWORD WINAPI functionVisit(LPVOID arg) // функція що Вона відповідає за дії клієнта, який відвідує барбера.
{
    /*
    Отримання ідентифікатора клієнта з аргументу, який передається функції.
    Значення ідентифікатора збільшується на 1, оскільки індексування починається з 0,
    але клієнти нумеруються починаючи з 1
    */
    int clientId = *static_cast<int*>(arg) + 1;

    EnterCriticalSection(&visitors_queue); // Вхід у критичну секцію visitors_queue, що блокує доступ до черги відвідувачів.

    if ((CHAIRS_NUM - visitors->getSize()) > 0) //Перевірка, чи є вільні місця в черзі для клієнтів.
    {
        EnterCriticalSection(&output); // входить в критичну секцію output, щоб мати доступ до виводу на екран.
        visitors->push(clientId); // додає клієнта до черги.
        cout << "Client " << clientId << " added to queue, queue size: " << visitors->getSize() << endl;
        visitors->printQueue();
        LeaveCriticalSection(&visitors_queue); // вихід з критичних секцій
        LeaveCriticalSection(&output);

        if (visitors->getSize() == 1)//Якщо це перший клієнт в черзі, розбуджуємо сплячого барбера
        {
            LeaveCriticalSection(&barber1);
        }
        EnterCriticalSection(&barber); // Захоплюємо критичну секцію barber, щоб виконати роботу барбера.

        EnterCriticalSection(&output); // критична секція виводу , виводимо на екран стан барбера
        cout << "Barber is working: " << clientId << endl;
        LeaveCriticalSection(&output); // виходимо з критичної секції
    }
    else // У випадку, якщо немає вільних стільців, клієнт не може встати в чергу
    {
        LeaveCriticalSection(&visitors_queue); // Звільняємо критичну секцію visitors_queue

        EnterCriticalSection(&output);
        // Виводимо повідомлення, що немає вільних місць і клієнт не може залишитися в черзі.
        cout << "No chairs, client " << clientId << " go away" << endl;
        LeaveCriticalSection(&output);
    }

    return true;
}
```

```cpp
int main()
{

    DWORD threadId; // Змінна для збереження ідентифікатора потоку.

    HANDLE barberThread; //  Дескриптор потоку для барбера
    HANDLE visitorsThreads[VISITORS_NUM]; //  Масив дескрипторів потоків для відвідувачів.
    int visitorsIds[VISITORS_NUM]; // Масив ідентифікаторів відвідувачів.


    visitors = new Queue(); //  Створення нового екземпляру черги

    // ініціалізація критичних секцій
    /*
    Коли критична секція ініціалізується за допомогою InitializeCriticalSection(),
    система створює спеціальний об'єкт критичної секції, який може бути використаний
    потоками для захоплення та звільнення доступу до критичної секції.
    Це забезпечує взаємовиключний доступ до коду або ресурсу,
    що захищається критичною секцією.
    */
    InitializeCriticalSection(&output);
    InitializeCriticalSection(&visitors_queue);
    InitializeCriticalSection(&barber);
    InitializeCriticalSection(&barber1);

    //Захоплення критичної секції barber1, щоб сплячий барбер залишався в стані очікування.
    EnterCriticalSection(&barber1);

    // Створення потоку для функції functionBarber, де NULL передається як параметр,
    // а &threadId вказує на змінну,
    // яка буде містити ідентифікатор потоку барбера.
    barberThread = CreateThread(NULL, 0, &functionBarber, NULL, 0, &threadId);

    for (int i = 0; i < VISITORS_NUM; ++i) //  Цикл для створення потоків для функції functionVisit для кожного відвідувача.
    {
        visitorsIds[i] = i; // Присвоєння ідентифікатора відвідувача.
        Sleep(1000 + rand() % 500); // симулюємо прибуття відвідувачів в різний час

        //Створення потоку для функції functionVisit з параметром, що вказує на ідентифікатор відвідувача.
        visitorsThreads[i] = CreateThread(NULL, 0, &functionVisit, &visitorsIds[i], 0, &threadId);
    }
    WaitForSingleObject(barberThread, INFINITE); // Очікування завершення потоку барбера.
    WaitForMultipleObjects(VISITORS_NUM, visitorsThreads, true, INFINITE); //  Очікування завершення всіх потоків відвідувачів.
```

```
CloseHandle(barberThread); // Закриття дескриптора потоку барбера.

for (int j = 0; j < VISITORS_NUM; ++j) //  Цикл для закриття дескрипторів потоків відвідувачів.
{
    CloseHandle(visitorsThreads[j]);
}

// видалення критичних секцій
DeleteCriticalSection(&output);
DeleteCriticalSection(&visitors_queue);
DeleteCriticalSection(&barber);

delete visitors;

return 0;
```